

EDINET API + XBRL データ解析 技術リサーチ

EDINET API v2 の仕様と取得設計

EDINET API v2 は、日付を起点に「その日に登録された提出書類の一覧（メタデータ付き）」を取得し、得られた `docID` を使って対象書類の ZIP（XBRL/iXBRL）や CSV（XBRL→CSV変換）を取得する、という **2段階フロー** が基本になります。 ¹

APIキーの取得手順と料金

API v2 利用にはアカウント登録と API キー発行が必要です。登録は EDINET 公式サイトから案内されており、ポップアップ許可の注意も記載されています。 ²

仕様書では、アカウント作成の過程で **SMS による確認コード（多要素認証）** を用い、サインイン後の画面で連絡先を登録すると API キーが表示される、と説明されています。 ³

- **料金（無料/有料）** : EDINET API 自体は公的システムの提供で、利用料金は発生しない運用として扱われるのが一般的（ただし、仕様書本文が「無料」と明記している箇所は確認できないため、ここは表現を抑えます）。
- **信頼度: 中**（公的提供である点は高い一方、料金明記の一次資料を本レポート内で確定できないため）

書類一覧 API `/api/v2/documents.json`

仕様書上、書類一覧 API は `date`（ファイル日付）等を指定して提出書類一覧（メタデータ）を返します。レスポンス例には `results` 配列と、各レコードに `docID`、`edinetCode`、`secCode`、`ordinanceCode`、`formCode`、`docTypeCode`、そして `xbrlFlag` / `pdfFlag` / `csvFlag` などの「有無フラグ」が含まれます。 ⁴

また、`fileDate` 指定の範囲は「（実運用上）最大で過去10年」相当に制約される旨が仕様書に記載されています。

TypeScript（Next.js/サーバー側）での最小呼び出し例（サーバー側で実行すること）：

```
type EdinetDocListResponse = {
  metadata: {
    status: string;
    message: string;
    processDateTime: string;
    resultset: { count: number };
  };
  results: Array<{
    docID: string;
    edinetCode: string | null;
    secCode: string | null;
    fileName: string | null;
    ordinanceCode: string | null;
    formCode: string | null;
```

```

docTypeCode: string | null;
periodStart: string | null;
periodEnd: string | null;
submitDateTime: string | null;
xbrlFlag: "0" | "1";
pdfFlag: "0" | "1";
csvFlag: "0" | "1";
// ...必要に応じて追加
}>;
};

export async function fetchEdinetDocsByDate(date: string, apiKey: string) {
  const url = new URL("https://api.edinet-fsa.go.jp/api/v2/documents.json");
  url.searchParams.set("date", date);
  // NOTE: type の意味（値域）は運用で要検証（後述）。
  url.searchParams.set("type", "2");
  url.searchParams.set("Subscription-Key", apiKey);

  const res = await fetch(url.toString(), { method: "GET" });
  if (!res.ok) throw new Error(`EDINET documents.json failed: ${res.status}`);
  return (await res.json()) as EdinetDocListResponse;
}

```

ブラウザ直叩きを避ける理由: 仕様書に「JavaScript でのクロスドメイン通信を許可しない」旨が明記されているため、Next.js の Route Handler / Server Action / Cron など **サーバー側**で呼ぶ前提にしてください。 ⁵

書類取得 API `/api/v2/documents/{docID}`

書類取得 API は `docID` をパスに置き、クエリパラメータ `type`（必要書類）で取得するファイル種別を指定します。仕様書は以下の対応を明記しています。 ⁶

- `type=1` : 提出本文書および監査報告書（ZIP）
- `type=2` : PDF（PDF）
- `type=3` : 代替書面・添付文書（ZIP）
- `type=4` : 英文ファイル（ZIP）
- `type=5` : CSV（ZIP、XBRL→CSV変換）

TypeScript の取得例:

```

export async function downloadEdinetDocument(
  docID: string,
  type: 1 | 2 | 3 | 4 | 5,
  apiKey: string,
): Promise<Buffer> {
  const url = new URL(`https://api.edinet-fsa.go.jp/api/v2/documents/${docID}`);
  url.searchParams.set("type", String(type));
  url.searchParams.set("Subscription-Key", apiKey);

  const res = await fetch(url.toString());
}

```

```
if (!res.ok) throw new Error(`EDINET documents/{docID} failed: ${res.status}`);
const ab = await res.arrayBuffer();
return Buffer.from(ab);
}
```

有価証券報告書のフィルタ条件（docTypeCode / ordinanceCode / formCode）

docTypeCode（書類種別）は仕様書のコードリストに掲載されており、たとえば以下が含まれます。

- 120 : 有価証券報告書
 - 130 : 四半期報告書
 - 140 : 半期報告書
 - 150 : 臨時報告書
 - 160 : 訂正有価証券報告書
- （ほか多数）

ordinanceCode（府令コード）も仕様書にコードリストがあり、010（開示府令）などが示されています。一方で **formCode（様式コード）**は「別紙（様式コードリスト）」参照として案内されており、API仕様書本文に一覧が直接載っていない構成です。

実務上は、コミュニティ実装で「有価証券報告書（通常方式）」を **ordinanceCode=010** + **formCode=030000** としてフィルタする例が多いです。⁷

- 信頼度: 中（実装例は複数あるが、公式の様式コードリスト（別紙）を本レポート内で直接突合できていないため）

フィルタは、コードだけに依存せずに **docTypeCode=120** に加えて **docDescription**（例: 「有価証券報告書」）や **xbrlFlag/csvFlag** を併用して安全側に倒すのが堅実です。⁸

レート制限の実態

仕様書内で「1分ごとに更新される」旨の記載があり、過剰ポーリングを避ける設計意図が読み取れます。ただし 「1秒あたり何回」「1日何回」等の明確な数値のレート制限は、仕様書本文だけでは確定できません。

- 信頼度: 低（数値を一次資料で確定不可）

推奨は「指数バックオフ + 同日分の差分取得 + 失敗時リトライのジッター + 取得済み docID の冪等処理」です（実装は後述）。

type=5（CSV形式）の実用性（API視点）

仕様書は **type=5** が「XBRLファイルをCSV形式に変換したZIP」であること、また **csvFlag=1** のときに取得できることを明記しています。⁹

CSVレイアウト詳細は「書類閲覧操作ガイド」参照、とされています。¹⁰

結論（API観点）: **csvFlag=1** の書類では **type=5** は「最短でパースに到達する近道」になりやすい（XMLや名前空間処理を大幅に減らせる）ため、StockManagement の初期実装として有力です（比較は後段）。

¹¹

ZIP 内構造と XBRL / iXBRL の落とし穴

ZIP内ディレクトリ構造（概観）

書類取得 API の `type=1` ZIPは、提出本文書・監査報告書・英文・添付等を用途別フォルダに分ける構造であることが仕様書に示されています。¹²

また、インラインXBRL提出の場合、EDINET画面からのXBRLダウンロードと異なり `XbrlSearchDLInfo.csv` が含まれないこと、ヘッダ・目次の一部は含まれないことも明記されています。¹⁰

さらに、EDINETの作成ツール操作ガイドには「YYYYMMDDhhmmssxxx / XBRL / PublicDoc」配下に、インラインXBRL（`*_header*.htm`、`*_honbun*.htm`）、`manifest_PublicDoc.xml`、そしてタクソノミ（`.xsd` / `.xml`）が作成される図示があります。¹³

TypeScript：ZIPからPublicDoc配下の候補を探索する最小例（後段の実処理の前処理として）：

```
export function isPublicDocXbrl(path: string): boolean {
    return /^(^\\|\\)XBRL\\PublicDoc\\.*\\.xbrl$/i.test(path);
}
export function isPublicDocInlineHtml(path: string): boolean {
    return /^(^\\|\\)XBRL\\PublicDoc\\.*_ixbrl\\.htm(l)?$/i.test(path) ||
        /^(^\\|\\)XBRL\\PublicDoc\\.*_(header|honbun)\\.htm(l)?$/i.test(path);
}
export function isPublicDocManifest(path: string): boolean {
    return /^(^\\|\\)XBRL\\PublicDoc\\manifest_PublicDoc\\.xml$/i.test(path);
}
```

iXBRL（.htm）と従来XBRL（.xbrl）の違いと出現頻度

EDINETの「インラインXBRL方式」は2013年度適用開始として整理されており、EDINETタクソノミの主要説明資料でもインラインXBRLを前提に章立てされています。¹⁴

また、API仕様書は「提出書類がインラインXBRL形式の場合でも、書類取得 API（type=1）でXBRLデータを取得でき、インスタンス形式のファイルが含まれる」と記述しています。¹⁰

- 近年の有価証券報告書では「iXBRL（HTML埋め込み）+ XBRLインスタンス（.xbrl）」の同梱が基本線、と見てよい。
- 信頼度: 高（制度・仕様書上の前提がインラインで、かつインスタンス同梱が明記）

名前空間プレフィックス（jpcrp_cor / jppfs_cor / jpdei_cor / jpigp_cor）の役割

EDINETタクソノミは大きく「内閣府令タクソノミ」「財務諸表本表タクソノミ（日本基準）」「国際会計基準タクソノミ（IFRS）」「DEIタクソノミ」に整理され、分割単位として `jpcrp` / `jppfs` / `jpigp` / `jpdei` が挙げられています。¹⁵

実務的には次の理解が最も役立ちます（StockManagementの「主要財務項目抽出」に直結）：

- `jppfs_*`（例: `jppfs_cor`）：日本基準（J-GAAP）財務諸表の本表（BS/PL/CF等）中心¹⁶
- `jpigp_*`（例: `jpigp_cor`）：IFRS財務諸表（本表+注記側要素も含む）中心¹⁷
- `jpdei_*`（例: `jpdei_cor`）：提出書類・提出者の基本情報（DEI）。名前空間宣言の例もガイドラインに明示。¹⁸

- `jpcrp_*` (例: `jpcrp_cor`) : 府令様式全体 (非財務も含む) や共通項目。分割単位表に有価証券報告書 (第三号様式等) を含むことが示される。 ¹⁹

contextRef による「連結/単体」「当期/前期」の判別

報告書インスタンス作成ガイドラインでは、コンテキストIDの典型パターン (`CurrentYearDuration` / `CurrentYearInstant` / `Prior1YearDuration` 等) と、個別を表す `NonConsolidatedMember` の付与・非付与が具体例付きで示されています。 ²⁰

重要ポイントは次の通りです。

- **連結**: シナリオ要素に `NonConsolidatedMember` を設定しない (= 連結は「何も付かない」ケースが多い) ²⁰
- **個別**: シナリオ要素に `NonConsolidatedMember` を設定する (例: `CurrentYearInstant_NonConsolidatedMember`) ²⁰

TypeScript : コンテキストを構造化して判定する型 (後段の抽出エンジンで使う) :

```
export type PeriodSpec =
  | { kind: "instant"; instant: string }
  | { kind: "duration"; startDate: string; endDate: string };

export type ContextInfo = {
  id: string;
  period: PeriodSpec;
  // true: 個別 (NonConsolidatedMember) / false: 連結 (デフォルト)
  nonConsolidated: boolean;
};
```

iXBRL の `scale` 属性の重要性和とし穴

インラインXBRLでは、表示単位 (百万円・千円など) に応じて `decimals` と `scale` を設定し、EDINETが iXBRL→XBRLインスタンスに変換する際に「指定された表示単位未満に0が設定される」ことが説明されています。 ²¹

また、負の値の記号 (括弧、△等) は **インラインXBRLタグの外に** 記載する、という規約も明示されています。 ²²

- 実装上の落とし穴:
- iXBRL (HTML) から直接数値を取るなら、`scale` と `sign` を必ず反映しないと桁がズれる。 ²¹
- `.xbrl` (インスタンス) から取るなら、多くの場合は **EDINET変換済みの正規化後数値** を扱える (ただしコンテキスト判定は必要)。 ²³

TypeScript : iXBRL 数値 (文字列+属性) → 整数に正規化する最小関数:

```
export function ixbrlToNumber(rawText: string, scale?: string, sign?: string): number | null {
  // "12,345" などの表示値 (タグ内) を想定
  const cleaned = rawText.replace(/,/g, "").trim();
  if (cleaned === "" || cleaned === "-" || cleaned === "-") return null;

  const n = Number(cleaned);
  if (!Number.isFinite(n)) return null;
```

```
const s = scale ? Number(scale) : 0;
const signed = (sign === "-" ? -1 : 1) * n;
return signed * Math.pow(10, s);
}
```

主要財務項目のタグ名カタログ（J-GAAP / IFRS / US-GAAP）

このセクションは「EDINETから主要指標を安定的に抜きたい」目的に合わせて、“**実務上の候補タグ集合**”としてまとめます。ベースはOSS `edinet-mcp` の `taxonomy.yaml`（会計基準差を吸収するために複数候補を列挙）で、EDINET特有のサフィックス（例: `...IFRSSummaryOfBusinessResults`）は「サフィックス剥がし」で吸収する方針が明示されています。 ²⁴

会計基準ごとの前提（特にUS-GAAP）

EDINET資料では、IFRS財務諸表は国際会計基準タクソノミを用いて詳細タグ付けも行う一方、**米国基準（US GAAP）の財務諸表は表ごとにテキストブロックで包括タグ付けすると記載されています。** ²⁵

つまり、US GAAP の場合、少なくとも「BS/PL/CF を数値Factとして機械抽出」するのは **制度上の提供形態と相性が悪い**（テキストブロックからの抽出＝HTML/PDF解析に近づく）点に注意してください。

- 信頼度: 高（一次資料に明記） ²⁵

カタログの提示方法

以下は、StockManagement が欲しい指標に限定して、**候補となる“ローカル名（Element local name）”**を列挙します（EDINET内では `jppfs_cor:` 等のプレフィックスが付きます）。`edinet-mcp` は会計基準差やサフィックス差を吸収するために複数候補を並べる設計です。 ²⁶

TypeScript：指標→候補タグ配列（ローカル名）定義例（レポート要求項目に絞った実装テンプレ）：

```
export type AccountingStandard = "JGAAP" | "IFRS" | "USGAAP";

export type MetricKey =
  | "revenue"
  | "operating_profit"
  | "net_income_parent"
  | "total_assets"
  | "equity"
  | "operating_cf"
  | "investing_cf"
  | "eps_basic"
  | "issued_shares"
  | "interest_bearing_debt"
  | "interest_expense"
  | "interest_paid_cf";

export const METRIC_TAGS: Record<MetricKey, Partial<Record<AccountingStandard, string[]>>> = {
  revenue: {
    JGAAP: ["NetSales"],
```

```

    IFRS: ["Revenue", "SalesRevenues", "TotalNetRevenues", "OperatingRevenues"],
    USGAAP: ["Revenues"], // ※US GAAPはFSがテキストブロック中心、ここは補助的
  },
  operating_profit: {
    JGAAP: ["OperatingIncome"],
    IFRS: ["OperatingProfit", "OperatingProfitLoss"],
  },
  net_income_parent: {
    JGAAP: ["ProfitLossAttributableToOwnersOfParent"],
    IFRS: ["ProfitAttributableToOwnersOfParent"],
  },
  total_assets: {
    JGAAP: ["TotalAssets"],
    IFRS: ["Assets"], // plain "Assets" = total assets
  },
  equity: {
    // J-GAAPは「純資産」、IFRSは「資本/持分」系で表現が異なる
    JGAAP: ["NetAssets"],
    IFRS: ["TotalEquity", "Equity"],
  },
  operating_cf: {
    JGAAP: ["CashFlowsFromOperatingActivities"],
    IFRS: ["CashFlowsFromUsedInOperatingActivities",
    "NetCashProvidedByUsedInOperatingActivities"],
  },
  investing_cf: {
    JGAAP: ["CashFlowsFromInvestingActivities"],
    IFRS: ["CashFlowsFromUsedInInvestingActivities",
    "NetCashProvidedByUsedInInvestingActivities"],
  },
  eps_basic: {
    // 利益/損失の両方に備える
    JGAAP: ["BasicEarningsPerShare", "BasicEarningsLossPerShare"],
    IFRS: ["BasicEarningsPerShare", "BasicEarningsLossPerShare"],
  },
  issued_shares: {
    // EDINET内で「株式数（株）、発行済株式、議決権の状況」に使われる代表例
    // ※対象セクション・軸（種類株式等）で揺れるため要追加調査
    JGAAP: ["NumberOfSharesIssuedSharesVotingRights"],
    IFRS: ["NumberOfSharesIssuedSharesVotingRights"],
  },
  interest_bearing_debt: {
    // 単一タグが保証されないため「合算」前提（短期+長期+社債+1年内返済予定など）
    JGAAP: [
      "ShortTermLoansPayable",
      "CurrentPortionOfLongTermLoansPayable",
      "LongTermLoansPayable",
      "BondsPayable",
    ],
    IFRS: [
      "ShortTermLoansPayable",

```

```

    "CurrentPortionOfLongTermLoansPayable",
    "LongTermLoansPayable",
    "BondsPayable",
  ],
},
interest_expense: {
  // taxonomy.yaml上では「支払利息」項目が定義されているが、要素名の列挙は運用で要検証
  JGAAP: ["InterestExpenses", "InterestExpense"],
  IFRS: ["InterestExpense", "FinanceCosts"],
  USGAAP: ["InterestExpense"],
},
interest_paid_cf: {
  // CF側の「利息の支払額」
  JGAAP: ["InterestPaid", "InterestExpensesPaid"],
  IFRS: ["InterestPaid", "InterestExpensesPaid"],
},
};

```

裏付け（本カタログの根拠）：- 売上高/営業利益/純利益（親会社帰属）候補は `taxonomy.yaml` の該当エントリに記載されています。 ²⁷

- 総資産・純資産（J-GAAP）/総資本（IFRS）相当も同様です。 ²⁸

- 営業CF/投資CF、および利息支払（CF側）は `cash_flow` セクションとして明示されています。 ²⁹

- 「株式数（株）、発行済株式、議決権の状況」に対応する要素として `NumberOfSharesIssuedSharesVotingRights` が確認できます。 ³⁰

- US GAAP 財務諸表がテキストブロック中心である前提はEDINET資料に明記です。 ²⁵

発行済株式数（issued_shares）の注意： - EDINET内では「主要な経営指標等の推移」「発行済株式、議決権の状況」「所有者別状況」など複数箇所に類似概念が存在し、軸（種類株式等）やコンテキスト（提出日/期末）も異なる設計です。 ³¹

- したがって、単一タグ固定でなく「対象セクション+コンテキスト+軸」を含めた取り方が必要です。

- **信頼度：中**（概念の複数箇所存在は資料で示唆、最適タグの確定は個別検証が必要）

Node.js / TypeScript での解析フルフロー

StockManagement の技術スタック（Next.js + Supabase + Vercel）を前提に、無料枠の制約下で「取得→展開→判定→抽出→保存」を回す実装パターンをまとめます。ブラウザ直叩き不可（クロスドメイン制約）なので、**必ずサーバー側（Route Handler / Cron）**で行います。 ⁵

ZIP展開ライブラリ選定（jszip / unzipper）

- `JSZip`：非同期APIは便利ですが、公式ドキュメントで **結果をメモリ上に保持する（大きいZIPで不利）** 旨が明示されています。 ³²
- `unzipper`：`Parse` / `ParseOne` による **ストリーミング解凍**を提供し、特定ファイルだけ抜き出す用途に向きます。 ³³

無料枠・メモリ制約を考えると、「**まず ParseOne で必要ファイルだけ抽出**」→「**抽出対象が小さければメモリに載せてXMLパース**」が堅いです。

XMLパーサ (fast-xml-parser)

`fast-xml-parser` は「XML→JSオブジェクト（バリデーションも可）」の軽量路線として NPM に記載があります。³⁴

(EDINET XBRL は要素・属性が多いので、属性保持設定などは別途調整が必要になります。)

iXBRLパーサ (cheerio)

Cheerio は HTML/XML を読み込み DOM 操作できるライブラリで、公式ドキュメントでも読み込み方法が整理されています。³⁵

iXBRL から `<ix:nonFraction>` 等を拾う場合に使えます（ただし `scale` / `sign` の反映が必須）。²²

数値正規化（全角数字・マイナス表記）

EDINETガイドラインでは「△や括弧はタグ外」といった表記規約があり、HTMLテキスト側を処理するなら正規化は必須です。³⁶

以下は「全角→半角」「括弧マイナス」「三角記号」「マイナス記号の多様体」を拾う実装例です。

```
const ZENKAKU_NUM = /[0-9]/g;

function zenkakuToHankaku(s: string): string {
  return s.replace(ZENKAKU_NUM, (ch) => String("0 1 2 3 4 5 6 7 8 9".indexOf(ch)));
}

export function parseJapaneseNumber(raw: string): number | null {
  let s = zenkakuToHankaku(raw)
    .replace(/\\s+/g, "")
    .replace(/,/g, "");

  if (s === "" || s === "-" || s === "—" || s === "—" || s === "—") return null;

  // よくある負値表現
  // (123) / (123) / △123 / ▲123 / -123 / -123
  let sign = 1;
  const paren = s.match(/^[^\( ]*(.+)[\)\( ]$/);
  if (paren) {
    sign = -1;
    s = paren[1];
  }
  if (/^[△▲]/.test(s)) {
    sign = -1;
    s = s.replace(/^[△▲]/, "");
  }
  if (/^[-- ----]/.test(s)) {
    sign = -1;
    s = s.replace(/^[-- ----]/, "");
  }

  const n = Number(s);
  if (!Number.isFinite(n)) return null;
}
```

```
return sign * n;
}
```

ZIP展開→ファイル判定→パース→値抽出（最小の全体像）

以下は「type=1（XBRL ZIP）から `.xbrl` を1つ抽出→XBRLインスタンスをパース→候補タグから数値を拾う」までの一連を“動く雛形”として提示します（実運用では複数ファイル・複数コンテキストに対応して拡張します）。

```
import unzipper from "unzipper";
import { XMLParser } from "fast-xml-parser";

// Buffer(zip) から最初にマッチした1ファイルだけ取り出す
async function extractFirstEntry(zip: Buffer, match: RegExp): Promise<Buffer> {
  const directory = await unzipper.Open.buffer(zip);
  const file = directory.files.find((f) => match.test(f.path));
  if (!file) throw new Error(`zip entry not found: ${match}`);
  const content = await file.buffer();
  return content;
}

type XbrlFact = { name: string; contextRef?: string; value?: string };

export async function extractMetricFromXbrlZip(zip: Buffer, candidates: string[]) {
  const xbrlBuf = await extractFirstEntry(zip, /^(^|\/)XBRL\/PublicDoc\/.*\.xbrl$/i);

  const parser = new XMLParser({
    ignoreAttributes: false,
    attributeNamePrefix: "@_",
    // EDINETは名前空間が多いので、運用で options を詰める
  });
  const doc = parser.parse(xbrlBuf.toString("utf-8"));

  // fast-xml-parser の出力形状はXML次第で揺れるため、ここは要調整。
  // 最小例として「xbrl配下の全要素を走査して name/ contextRef / value」を拾う想定。
  // 実運用は「QName合致」「context精査」「unit精査」「nil対応」までやる。
  const facts: XbrlFact[] = []; // <- ここに抽出口ジックを実装（後述のマッピング戦略で強化）

  // 候補タグを優先順で引いて最初に見つかったものを返す例
  for (const localName of candidates) {
    const hit = facts.find((f) => f.name === localName && f.value != null);
    if (hit) return parseJapaneseNumber(hit.value!);
  }
  return null;
}
```

この雛形を「CSV（type=5）」に寄せると、XML走査の代わりに「行（Fact）走査」になり実装が単純化します（比較・推奨は後段）。 37

勘定科目マッピング戦略とデータ構造

「主要財務項目を自動抽出」する工程で最もコストがかかるのは、**会計基準差 + EDINET特有のサフィックス + 企業拡張（提出者別タクソノミ）**を踏まえたマッピングです。edinet-mcp はこの課題に対して「taxonomy.yaml に正規形ラベルと候補要素名を列挙し、サフィックスを剥がして一致させる」方針を採っています。²⁴

科目名の揺れ（具体例 10+）

実際に「同じ意味でも表現が変わる」典型例（XBRL要素名/ラベルの観点）を、主要指標に絞って列挙します。

- 売上高: NetSales (J-GAAP)、Revenue (IFRS)、Revenues (US系バリエーション)、SalesRevenues (IFRS系)、OperatingRevenues (IFRS系・キー財務データ側)³⁸
- 営業利益: OperatingIncome (J-GAAP)、OperatingProfit / OperatingProfitLoss (IFRS系)³⁹
- 親会社株主に帰属する当期純利益: ProfitLossAttributableToOwnersOfParent (J-GAAP)、ProfitAttributableToOwnersOfParent (IFRS)⁴⁰
- 当期純利益（総称）: ProfitLoss (J-GAAP)、ProfitForThePeriod (IFRS)⁴⁰
- 総資産: TotalAssets (J-GAAP)、Assets (IFRSで総資産として使われる)⁴¹
- 自己資本/純資産: NetAssets (J-GAAP)、TotalEquity / Equity (IFRS)⁴²
- 営業CF: CashFlowsFromOperatingActivities (J-GAAP)、NetCashProvidedByUsedInOperatingActivities (IFRS詳細CF) など⁴³
- 投資CF: CashFlowsFromInvestingActivities (J-GAAP)、NetCashProvidedByUsedInInvestingActivities (IFRS詳細CF) など⁴³
- EPS: BasicEarningsPerShare / BasicEarningsLossPerShare (損失時の別要素に備える)⁴⁴
- 有利子負債（合算前提）: ShortTermLoansPayable、LongTermLoansPayable、BondsPayable、CurrentPortionOfLongTermLoansPayable などの組み合わせ⁴⁵
- 支払利息（CF側）: InterestPaid / InterestExpensesPaid (利息支払の表現ゆれを考慮)⁴⁶

上記の「揺れ」は、会計基準差だけでなく EDINETが要素名末尾に付加するサフィックス差（例: ...IFRSSummaryOfBusinessResults）でも増幅します。edinet-mcp はこの末尾を剥がして寄せる設計を明示しています。⁴⁷

会計基準の判定 (jpdei_cor:AccountingStandardsDEI)

DEI（提出書類・提出者の基本情報）はインスタンス内で参照しやすいことがガイドラインに説明されています。⁴⁸

jpdei_cor 名前空間自体はガイドラインで宣言例が示されます。⁴⁹

AccountingStandardsDEI という要素名で会計基準を読む実装例（Arelle利用例）が公開されており、値として“Japan GAAP”等が入る例が示されています。⁵⁰

- 信頼度: 中（要素名は実装例で確認できるが、「全書類で必ず存在」「値の正規形が常に同一」までは一次資料で断定できないため）

TypeScript: 会計基準を読むインターフェース設計 (CSV/XBRL共通):

```
export type AccountingStandardDetected = "JapanGAAP" | "IFRS" | "USGAAP" | "JMIS" | "Unknown";

export type DeiInfo = {
  accountingStandard?: string; // 元文字列を保持 (例: "Japan GAAP")
}
```

```
};

export function normalizeAccountingStandard(raw?: string): AccountingStandardDetected {
  if (!raw) return "Unknown";
  const s = raw.toLowerCase();
  if (s.includes("japan")) return "JapanGAAP";
  if (s.includes("ifrs")) return "IFRS";
  if (s.includes("us")) return "USGAAP";
  if (s.includes("jmis")) return "JMIS"; // 修正国際基準
  return "Unknown";
}
```

優先順位付きフォールバック検索（設計パターン）

推奨パターンは「候補リストを“意味の近さ”順に並べ、最初にヒットした値を採用」＋「コンテキスト優先（連結・当期）」です。コンテキストIDの規約（`CurrentYear...` / `Prior1Year...` / `...NonConsolidatedMember`）はガイドラインに具体例があります。 20

TypeScript：候補タグ×コンテキスト優先の擬似コード：

```
type Fact = { localName: string; contextId: string; value: number | null };

function contextScore(ctx: ContextInfo): number {
  // 連結を優先 (= nonConsolidated false を高得点)
  const base = ctx.nonConsolidated ? 0 : 100;

  // 当期/前期の優先 (例: CurrentYear を優先)
  const isCurrent = ctx.id.includes("CurrentYear");
  const isPrior1 = ctx.id.includes("Prior1Year");
  return base + (isCurrent ? 20 : 0) + (isPrior1 ? 10 : 0);
}

export function pickBestFact(facts: Fact[], contexts: Record<string, ContextInfo>, candidates: string[]) {
  const byCandidate = facts.filter(f => candidates.includes(f.localName) && f.value != null);
  byCandidate.sort((a, b) => (contextScore(contexts[b.contextId]) - contextScore(contexts[a.contextId])));
  return byCandidate[0]?.value ?? null;
}
```

edinet-mcp の taxonomy.yaml アプローチ評価

`edinet-mcp` は「会計基準差を吸収して正規ラベルでアクセスできる」「マッピングは YAML に集約」「EDINETサフィックスは剥がして一致」などを明示しています。 24

- 長所: マッピングの追加が**コード変更なし**で可能、テストしやすい、差分レビューが容易。 24
- 短所: サフィックス剥がしは便利だが、要素名の衝突や誤一致リスクがある（特に企業拡張が多い領域）。
- 信頼度: 中（設計上の推論：一般に末尾正規化は衝突可能性を持つ）

推奨マッピングテーブルのデータ構造（Supabase保存を想定）

YAML/JSON で保持しつつ、DB側にも「採用したタグ」「採用したコンテキスト」「元Fact値」を残すと、後で検証・再計算しやすくなります。

```
export type CanonicalMetric = {
  key: MetricKey;
  labelJa: string;
  labelEn?: string;
  candidates: Partial<Record<AccountingStandard, string[]>>;
};

export type ExtractionResult = {
  docID: string;
  metricKey: MetricKey;
  value: number | null;
  unit?: string; // JPY / shares / pure ...
  contextId?: string;
  matchedLocalName?: string; // 実際に一致した要素名（サフィックス除去前/後を別で持ってもよい）
  confidence: "high" | "medium" | "low";
};
```

既存 OSS の評価（適用可能性）

StockManagement（個人開発、無料枠運用）では「実装コストの低さ」と「壊れにくさ（メンテしやすさ）」が最優先になります。その観点で主要 OSS を評価します。

edinet-mcp（Python、taxonomy.yaml）

会計基準差を吸収して「`stmt["売上高"]`」のように正規ラベルで取れる」こと、マッピングが `taxonomy.yaml` に定義され、サフィックス剥がしで解決する設計であることが明記されています。 ²⁴

- 適用: 設計の参考（マッピング思想）として最重要。ただし Next.js/TypeScript 直実装にそのまま組み込むには言語が異なる。

- 推奨度: 高（思想と taxonomy 一覧がそのまま転用可能）

edinet-xbrl（Python）

`edinet_xbrl` は EDINET XBRL の Python パーサとして公開されています。 ⁵¹

- 適用: 解析ロジックの参考になるが、TypeScript 移植が必要。

- 推奨度: 中（すでに TypeScript 移植として `edinet-ts` が存在）

Arelle（XBRL国際標準パーサ、Python）

Arelle は GUI/CLI/Python API/Web Service API を備え、Inline XBRL v1.1 など XBRL 関連仕様への対応をうたい、XBRL International の Validating Processor 認証にも言及があります。 ⁵²

また、Inline XBRL Document Set (IXDS) プラグインが「JP FSA の manifest ファイルのようなもの」を扱える、とコード上で説明されています。 ⁵³

- 適用: 「正しさ優先」のときに強いが、サーバーレス無料枠で回すには重い（起動時間・依存の重さ・処理時

間)。

- 推奨度: 中 (検証/テスト用には高、プロダクション無料枠ETLには慎重)

dexter-jp / edinetdb (TypeScript)

`dexter-jp` は TypeScript ベースで、データソースとして EDINET DB API を用いるエージェントとして公開されています。 ⁵⁴

- 適用: 「EDINET生XBRL解析」用途ではなく、**データベンダ/API利用**寄り。StockManagement が目指す「EDINET APIから自動抽出」とはレイヤが異なる。
- 推奨度: 低～中 (目的が“自前パイプライン”なら直接の部品にはなりにくい)

edinet-ts (TypeScript)

`edinet-ts` は EDINET API v2 を利用して XBRLをダウンロード・解析する TypeScript ライブラリで、`edinet_xbrl` のロジックを移植した旨が記載されています。 ⁵⁵

- 適用: StockManagement の技術スタックに最も近い。
- 推奨度: 高 (ただし、プロジェクト最適化のため「必要部分だけ参照・薄く取り込む」方が保守しやすい)

サーバーレス制約と運用設計 (Vercel / Supabase)

ここは「月額¥0」を守るうえで最も重要です。先に結論を書くと、**Vercel単体で“多数社×大量書類”のXBRLパースを同期HTTPで完結させるのは難しく、Cron+分割処理(キュー/バックグラウンド)設計が必要です。** ⁵⁶

Vercel サーバーレス関数の制約 (Hobby前提)

Vercel の Hobby プランは「Vercel Function maximum duration はデフォルト 10秒、設定で最大60秒」という形で掲載されています。 ⁵⁷

また、Functions の制限表 (Node.js runtime) には最大durationや最大メモリ等が整理されています。 ⁵⁸

- ・「10秒でXBRL取得+ZIP展開+パース」は、**単発なら条件次第で可能**だが、ネットワーク揺らぎ・ZIPサイズ次第で落ちやすい。
- ・**信頼度: 中** (時間制限そのものは一次資料で高いが、“可能か”は書類サイズ分布に依存)
- ・Hobby でも `maxDuration` を上げられるが、無料枠設計としては「短時間で終わる単位に分割」が前提になる。 ⁵⁶

TypeScript : Vercel (Next.js Route Handler) 側で maxDuration を明示する例 (Next.jsの実行環境に合わせて調整) :

```
// app/api/edinet/sync/route.ts
export const runtime = "nodejs";
export const maxDuration = 60; // Hobbyで上げられる範囲に合わせる (上限はプラン依存)

export async function POST() {
  // ここでは「キューに積む」「DBにジョブ登録」など短時間で終える
  return new Response("queued", { status: 202 });
}
```

バックグラウンド処理・分割処理

Vercel はタイムアウト対策として **waitUntil / after (Next.js)** によるバックグラウンド処理を案内しています。 ⁵⁹

また、durable なイベント処理として Vercel Queues (Beta) が案内されています。 ⁶⁰

定期起動は Vercel Cron Jobs で実現できます。 ⁶¹

Supabase Edge Functions の実用性（無料枠）

Supabase Edge Functions の制限として、Freeプランで **最大150秒 (wall clock)**、メモリ **256MB**、CPU time などがドキュメントに明記されています。 ⁶²

さらに「Background tasks」文脈でも無料の上限が 150秒であることがブログで説明されています。 ⁶³

- XBRL ZIP の扱いは「256MBメモリ」に引っかかるリスクがあるため、**ストリーミング展開（必要ファイルのみ抽出）** が本質的に重要。 ⁶⁴
- **信頼度: 中**（制限値は高、処理可能性はZIPサイズ分布依存）

有報ZIPのサイズ分布（平均/最大）

一次資料ベースで「平均/最大」を確定するデータを本レポート内で提示できません。

- **信頼度: 低**

代わりに、運用開始後すぐに **Content-Length** を記録して自前で分布を作るのが最も確実です（無料ででき、設計判断に直結します）。

```
export type ZipSizeLog = {
  docID: string;
  type: 1 | 5;
  contentLength?: number;
  fetchedAt: string;
};

export async function downloadWithSize(docID: string, type: 1 | 5, apiKey: string) {
  const url = new URL(`https://api.edinet-fsa.go.jp/api/v2/documents/${docID}`);
  url.searchParams.set("type", String(type));
  url.searchParams.set("Subscription-Key", apiKey);

  const res = await fetch(url.toString());
  const len = res.headers.get("content-length");
  const contentLength = len ? Number(len) : undefined;

  const buf = Buffer.from(await res.arrayBuffer());
  return { buf, contentLength };
}
```

CSV (type=5) と XBRL パースの比較と推奨

CSV (type=5) の形式

API仕様書は、`type=5` が「提出本文書および監査報告書のXBRLファイルをCSV形式に変換したZIP」であること、ZIP内に `XBRL_T0_CSV` フォルダがあること、CSVレイアウトは操作ガイド参照であることを示しています。¹⁰

加えて、XBRL→CSV変換ツールの操作ガイドでは「拡張子はcsvだが区切りはカンマではなくタブ（TSV相当）」と説明されています。⁶⁵

実務記事では文字コードが UTF-16LE かつタブ区切りである点、ZIP内に `jpcrp_...csv` 等が入る点が述べられています。⁶⁶

- 文字コードUTF-16LE: 信頼度: 中（コミュニティ実装に依拠）

CSV の列構造とパース方法

変換ツールガイドでは、少なくとも「コンテキストID」「期間・時点」「ユニットID」「単位」「値」など、XBRL Fact をフラット化した列群が示されています。⁶⁵

TypeScript : UTF-16LE + TSV想定 of 読み取り (概念コード) :

```
import iconv from "iconv-lite";

export function decodeEdinetCsv(buffer: Buffer): string {
  // UTF-16LE想定 (運用でBOM検知等も追加)
  return iconv.decode(buffer, "utf-16le");
}

export function parseTsvLines(text: string): string[][] {
  return text
    .split(/\r?\n/)
    .filter((l) => l.length > 0)
    .map((l) => l.split("\t"));
}
```

CSVで取得できる範囲 / できない範囲

- CSVは「XBRLをフラット化したもの」なので、XBRLインスタンスに含まれるFactを広く拾える方向。³⁷
- 一方で、US GAAPのように「財務諸表がテキストブロック中心」の場合は、そもそも数値Factが存在しない (= CSVにしても数値が出ない/限定的) 可能性が高い。²⁵
- また、CSVは「タクソノミ関係 (計算リンク、表示リンク等)」を活用する解析には向かず、必要ならXBRLパッケージ (`.xsd` / リンクベース) 側を別途読む必要があります。⁶⁷
- 信頼度: 中 (関係リンク活用にXSD/リンクベースが必要という一般原理 + EDINET資料の構成説明に基づく推論)

推奨：どちらから始めるべきか

StockManagement の目的（主要財務項目の自動抽出→保存）に対しては、実装・運用の現実性から **CSV (type=5) 先行**が合理的です。理由は以下です。

- APIとして `type=5` が正式に提供され、`csvFlag` で可否も判定できる。 ¹¹
- XML/名前空間/iXBRLの `scale/sign` などの落とし穴を初期段階で回避できる。 ⁶⁸
- 無料枠サーバーレスでは「短時間・小メモリ」が前提になりやすく、TSVの方が処理が軽い。 ⁶⁹

ただし、`csvFlag=0` の書類（または将来的な仕様差）にも備え、**XBRL (.xbrl) パースをフォールバックとして持つ**のが長期的には安定します。 ⁷⁰

StockManagementプロジェクトへの推奨実装ロードマップ

最後に、Next.js + Supabase + Vercel（月額¥0）という制約の中で、最短で「動く」→「壊れにくい」へ進む実装順を提示します。

フェーズ設計（要点）

- EDINET API はブラウザから呼べない前提なので、すべてサーバー側に閉じる。 ⁵
- Vercel Hobby のデフォルト10秒は短いので、「Cron起動→ジョブ登録→分割処理」が必須。 ⁷¹
- Supabase Edge Functions（Free 150s/256MB）は「重い処理の受け皿」として有力だが、ZIPサイズ次第でメモリ注意。 ⁷²

推奨ステップ

Step A: メタデータ収集の自動化（最小）

Vercel Cron Jobs で 1日数回（または平日毎日） `/api/edinet/pull` を叩き、`/documents.json` の `results` を Supabase に保存（`docID` を主キーに冪等）。`xbrlFlag/csvFlag` も保存する。 ⁷³

```
// Supabaseに保存するdocメタの最小形
export type StoredDocMeta = {
  docID: string;
  fileDate: string;
  submitDateTime?: string | null;
  docTypeCode?: string | null;
  ordinanceCode?: string | null;
  formCode?: string | null;
  csvFlag: "0" | "1";
  xbrlFlag: "0" | "1";
};
```

Step B: 有価証券報告書の抽出条件を二重化

まずは `docTypeCode=120`（有価証券報告書）を軸に、必要なら `ordinanceCode/formCode` を補助条件として扱う（`formCode`は別紙リストで最終確認）。 ⁷⁴

- 信頼度: 中（`formCode`の確定が別紙依存）

Step C: type=5 (CSV) で主要指標の抽出を先に完成させる

csvFlag=1 の doc に限定して type=5 ZIP を取得→TSVをFact化→METRIC_TAGS の候補タグで値を拾う
→ ExtractionResult として保存。 75

Step D: コンテキスト戦略を“連結優先”で固定

NonConsolidatedMember の有無で単体/連結を切り分け、基本は連結を優先して採用する（投資分析ではこの方が比較しやすい）。 76

Step E: XBRL (type=1) をフォールバックとして追加

csvFlag=0 でも xbrlFlag=1 の可能性があるため、フォールバックとして .xbrl を抽出して同じ正規化口ジックへ流す。 70

Step F: 実行基盤の最適化（無料枠に合わせる）

- Vercel側: Cron→ジョブ登録までを60秒以内に収める（maxDurationを調整）。 57
- 重い処理: Supabase Edge Functions に寄せ、150秒以内に「1 docID を処理」できるように分割。 77
- さらに分割が必要なら、Vercelの waitUntil/after や Queues を併用して「小さな仕事」にする。 78

Step G: マッピングの保守性を上げる

edinet-mcp の taxonomy.yaml のように「マッピングをデータで持つ」方式へ寄せ、差分追加をテストで担保する。 24

最終的な到達像（無料枠で現実的）

- 1) Vercel Cron → 2) SupabaseにdocIDジョブを積む → 3) Edge Functionsで type=5 を中心に1件ずつ処理 → 4) 失敗はリトライ／サイズログを貯めて設計更新、というループが「壊れにくい最短路」です。 79

1 3 4 5 6 8 9 10 11 12 37 70 75 <https://disclosure2dl.edinet-fsa.go.jp/guide/static/disclosure/download/ESE140206.pdf>

<https://disclosure2dl.edinet-fsa.go.jp/guide/static/disclosure/download/ESE140206.pdf>

2 <https://disclosure2.edinet-fsa.go.jp/WEEK0010.aspx>

<https://disclosure2.edinet-fsa.go.jp/WEEK0010.aspx>

7 74 <https://qiita.com/YoshitakaOkada/items/877a331a450a8f27f985>

<https://qiita.com/YoshitakaOkada/items/877a331a450a8f27f985>

13 <https://disclosure2dl.edinet-fsa.go.jp/guide/static/submit/download/ESE140107.pdf>

<https://disclosure2dl.edinet-fsa.go.jp/guide/static/submit/download/ESE140107.pdf>

14 15 16 17 19 25 31 67 https://www.fsa.go.jp/search/20211109/1b-1_GaiyoSetsumei.pdf

https://www.fsa.go.jp/search/20211109/1b-1_GaiyoSetsumei.pdf

18 49 https://www.fsa.go.jp/search/20251111/2a-1_TaxonomyGuide.pdf

https://www.fsa.go.jp/search/20251111/2a-1_TaxonomyGuide.pdf

20 21 22 23 36 48 68 76 https://www.fsa.go.jp/search/20241112/2b-1_InstanceGuide.pdf

https://www.fsa.go.jp/search/20241112/2b-1_InstanceGuide.pdf

24 47 <https://github.com/ajtgjmdjp/edinet-mcp>

<https://github.com/ajtgjmdjp/edinet-mcp>

26 27 28 29 38 39 40 41 42 43 44 45 46 https://github.com/ajtgjmdjp/edinet-mcp/blob/main/src/edinet_mcp/data/taxonomy.yaml
https://github.com/ajtgjmdjp/edinet-mcp/blob/main/src/edinet_mcp/data/taxonomy.yaml

30 <https://fractalexperience.com/xbrl/web/?h=3e0b8b8688a6081e0e77da0b356a0a1ea57df39d>
<https://fractalexperience.com/xbrl/web/?h=3e0b8b8688a6081e0e77da0b356a0a1ea57df39d>

32 <https://stuk.github.io/jszip/documentation/limitations.html>
<https://stuk.github.io/jszip/documentation/limitations.html>

33 64 <https://www.npmjs.com/package/unzipper>
<https://www.npmjs.com/package/unzipper>

34 <https://www.npmjs.com/package/fast-xml-parser>
<https://www.npmjs.com/package/fast-xml-parser>

35 <https://cheerio.js.org/>
<https://cheerio.js.org/>

50 <https://speakerdeck.com/miya8/pyconjp2020-5-0828-1450>
<https://speakerdeck.com/miya8/pyconjp2020-5-0828-1450>

51 https://github.com/BufferCode/edinet_xbrl
https://github.com/BufferCode/edinet_xbrl

52 <https://arelle.readthedocs.io/>
<https://arelle.readthedocs.io/>

53 <https://github.com/Arelle/Arelle/blob/master/arelle/plugin/inlineXbrlDocumentSet.py>
<https://github.com/Arelle/Arelle/blob/master/arelle/plugin/inlineXbrlDocumentSet.py>

54 <https://github.com/edinetdb/dexter-jp>
<https://github.com/edinetdb/dexter-jp>

55 <https://github.com/oharato/edinet-ts>
<https://github.com/oharato/edinet-ts>

56 57 69 71 <https://vercel.com/docs/plans/hobby>
<https://vercel.com/docs/plans/hobby>

58 <https://vercel.com/docs/functions/limitations>
<https://vercel.com/docs/functions/limitations>

59 78 <https://vercel.com/kb/guide/what-can-i-do-about-vercel-serverless-functions-timing-out>
<https://vercel.com/kb/guide/what-can-i-do-about-vercel-serverless-functions-timing-out>

60 <https://vercel.com/docs/queues>
<https://vercel.com/docs/queues>

61 73 79 <https://vercel.com/docs/cron-jobs>
<https://vercel.com/docs/cron-jobs>

62 72 77 <https://supabase.com/docs/guides/functions/limits>
<https://supabase.com/docs/guides/functions/limits>

63 <https://supabase.com/blog/edge-functions-background-tasks-websockets>
<https://supabase.com/blog/edge-functions-background-tasks-websockets>

65 <https://www.fsa.go.jp/search/20130917/07.pdf>
<https://www.fsa.go.jp/search/20130917/07.pdf>

66 <https://zenn.dev/htera/articles/3d8f7e0388fff8>
<https://zenn.dev/htera/articles/3d8f7e0388fff8>